

Laporan Pengembangan Aplikasi Mahasiswa (MVC Sederhana)

Laporan ini merinci struktur dan implementasi fitur-fitur utama dalam aplikasi manajemen data mahasiswa yang dibangun menggunakan arsitektur Model-View-Controller (MVC) sederhana.

1. Struktur Proyek (MVC)

Proyek ini diorganisir menggunakan pola MVC untuk memisahkan logika bisnis, presentasi, dan akses data.

- `app/Models/`: Berisi `MahasiswaModel.php` yang menangani semua interaksi langsung dengan database PostgreSQL menggunakan PDO.
- `app/Views/`: Berisi template tampilan (UI).
- `auth/`: Halaman Login dan Register.
- `admin/`: Halaman Overview, Profile, dan Data Mahasiswa.
- `layout/`: Header dan Footer yang digunakan secara konsisten di seluruh aplikasi.
- `app/Controllers/`: Bertindak sebagai perantara.
- `AuthController.php`: Menangani proses autentikasi (Login, Register, Logout).
- `AdminController.php`: Menangani halaman setelah login (Overview, Profile, List Mahasiswa).
- `public/index.php`: Front Controller. Semua request masuk melalui file ini, yang kemudian menentukan controller mana yang akan dijalankan berdasarkan parameter route.
- `config/database.php`: Menyimpan konfigurasi koneksi database.

2. Implementasi Modul Tugas

Sesuai dengan instruksi tugas, berikut adalah rincian implementasi untuk setiap halaman:

A. Modul Login

- Controller: `AuthController::showLogin()` (menampilkan form) dan `AuthController::login()` (proses validasi).
- Model: `MahasiswaModel::login()` mencocokkan username dan memverifikasi hash password.
- View: `app/Views/auth/login.php`.
- Fitur: Melakukan pengecekan `password_verify` terhadap hash BCrypt di database. Jika berhasil, aplikasi menjalankan `session_regenerate_id(true)` lalu menyimpan ID user dalam `$_SESSION`.

Contoh kode login:

```
public function login(): void
{
    $user = trim($_POST['username'] ?? '');
    $pass = $_POST['password'] ?? '';
```

```

$row = $this->model->login($user, $pass);
if (!$row) {
    $this->showLogin('Login gagal. Username atau password salah.');
```

```

    return;
}

session_regenerate_id(true);
$_SESSION['user_id'] = (int) $row['id'];
$_SESSION['username'] = $row['username'];
$this->storeLoginCookies($row['username']);

header('Location: index.php?route=overview');
exit;
}

```

B. Modul Cookies

- Implementasi: Setelah login berhasil, aplikasi menyimpan cookie last_login_username dan last_login_at.
- Pemakaian: Cookie ditampilkan kembali pada halaman login dan overview untuk membuktikan bahwa data cookie berhasil tersimpan di browser.
- Tujuan: Memenuhi materi praktikum tentang penggunaan cookie untuk menyimpan data sederhana di sisi client.

Contoh kode cookie:

```

private function storeLoginCookies(string $username): void
{
    $expires = time() + 604800;

    setcookie('last_login_username', $username, $expires, '/', '', false, true);
    setcookie('last_login_at', date('Y-m-d H:i:s'), $expires, '/', '', false, true);
}

```

C. Modul Register

- Controller: AuthController::register().
- Model: MahasiswaModel::register() dan MahasiswaModel::isUsernameTaken().
- View: app/Views/auth/register.php.
- Fitur: Validasi input (wajib diisi, panjang karakter), pengecekan username unik, dan hashing password sebelum disimpan ke database.

Contoh kode register:

```

public function register(): void
{
    $data = [
        'nama' => trim($_POST['nama'] ?? ''),
        'nrp' => trim($_POST['nrp'] ?? ''),
        'alamat' => trim($_POST['alamat'] ?? ''),
        'ttl' => trim($_POST['ttl'] ?? ''),
        'email' => trim($_POST['email'] ?? ''),
        'nohp' => trim($_POST['nohp'] ?? ''),
        'username' => trim($_POST['username'] ?? ''),
        'password' => $_POST['password'] ?? '',
    ];
}

```

```

foreach (['nama', 'nrp', 'alamat', 'ttl', 'email', 'nohp', 'username', 'password'] as $field) {
    if ($data[$field] === '') {
        $this->showRegister('Semua field wajib diisi.', $data);
        return;
    }
}

if ($this->model->isUsernameTaken($data['username'])) {
    $this->showRegister('Username sudah digunakan.', $data);
    return;
}

$data['password'] = password_hash($data['password'], PASSWORD_BCRYPT);

if (!$this->model->register($data)) {
    $this->showRegister('Registrasi gagal. Coba lagi.', $data);
    return;
}

header('Location: index.php?route=login');
exit;
}

```

D. Modul Overview

- Controller: AdminController::overview().
- View: app/Views/admin/overview.php.
- Fitur: Menampilkan ringkasan informasi setelah user berhasil login, termasuk username aktif dan data cookie login terakhir. Dilengkapi dengan pengecekan requireLogin() untuk memastikan hanya user terautentikasi yang bisa mengakses.

Contoh kode overview:

```

public function overview(): void
{
    $this->requireLogin();
    $username = $_SESSION['username'] ?? '';
    $rememberedUsername = $_COOKIE['last_login_username'] ?? '';
    $lastLoginAt = $_COOKIE['last_login_at'] ?? '';

    require __DIR__ . '/../Views/admin/overview.php';
}

```

E. Modul Profile

- Controller: AdminController::profile().
- Model: MahasiswaModel::getProfile(\$id).
- View: app/Views/admin/profile.php.
- Fitur: Menampilkan data detail profil mahasiswa berdasarkan ID user yang sedang login. Parameter id dari URL tidak lagi digunakan agar data profil user lain tidak bisa dibuka sembarangan.

Contoh kode profile:

```

public function profile(): void
{

```

```

$this->requireLogin();
$id = (int) $_SESSION['user_id'];
$profile = $this->model->getProfile($id);

require __DIR__ . '/../Views/admin/profile.php';
}

```

F. Modul Data Mahasiswa

- Controller: AdminController::mahasiswaList().
- Model: MahasiswaModel::getAllMahasiswa().
- View: app/Views/admin/mahasiswa.php.
- Fitur: Mengambil daftar seluruh mahasiswa yang terdaftar di database dan menampilkannya dalam format tabel. Halaman ini difokuskan sebagai daftar data, bukan jalur untuk membuka profil user lain.

Contoh kode model daftar mahasiswa:

```

public function getAllMahasiswa(): array
{
    $sql = "SELECT id, nrp, nama FROM {$this->schema}.mahasiswa ORDER BY id ASC";
    $stmt = $this->db->query($sql);

    return $stmt->fetchAll();
}

```

3. Detail Teknis

- Database: PostgreSQL dengan skema khusus (mahasiswa).
- Autentikasi: Menggunakan PHP Session standar yang diperkuat dengan regenerasi session ID setelah login.
- Cookies: Menggunakan cookie HTTP-only untuk menyimpan username login terakhir dan waktu login terakhir selama 7 hari.
- Styling: CSS eksternal di public/assets/css/style.css untuk memastikan tampilan yang rapi dan konsisten.
- Keamanan: Penggunaan password_hash() dan password_verify() untuk manajemen password, serta PDO prepared statements untuk mencegah SQL Injection.